IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Project manuscript prepared 24 October 2026

Working manuscript — no DOI assigned

AeroLab: A Model-Based, Digital-Twin-Inspired Residual Monitoring Testbed with Sampled-Data LQR Control and Live Telemetry on an ESP32 Platform

Manav Sharma

School of Electronics Engineering, Vellore Institute of Technology, Vellore, India
Corresponding author: Manav Sharma.

This work is a cyber-physical demonstrator; closed-loop control and digital-twin fault diagnosis are evaluated in sampled-data simulation, while the ESP32 rig provides live multi-sensor telemetry and open-loop flight-surface actuation.

ABSTRACT—AeroLab adapts the residual-based digital-twin fault-diagnosis framework of Araujo et al. to a low-cost embedded testbed built on an ESP32. A second-order flight-surface model is discretized by zero-order hold at 20 Hz, a discrete LQR gain and a pole-placed observer are synthesized in MATLAB and exported to firmware, and a noiseless model twin runs in parallel with the plant trajectory. Their difference feeds a fixed-threshold test and a two-sided CUSUM detector, while ultrasonic, inertial, thermal, and gas channels drive a prioritized alarm logic visualized across browser, MATLAB, and Python scopes. Control and detector benchmarks are simulation-based: the plant parameters are assumed, and the hobby servo operates in open-loop PWM actuation without an internal position encoder. Hence, closed-loop state feedback and CUSUM residuals are evaluated around the synchronized sampled-data mathematical model, making this a model-based monitoring testbed rather than a flight-certified hardware twin. In simulation, CUSUM detects persistent offsets down to 0.03 (1.5x reference) with zero false alarms at nominal noise, whereas fixed thresholds require offsets above 0.10. Bounded live hardware checks over USB (460,800 baud) confirm fresh dual-channel ultrasonic ranging (US1 altitude and US2 servo radar) and valid multi-tier alarm transitions. Host C++ unit tests execute and pass identically to the MATLAB design before firmware deployment.

INDEX TERMS—embedded control, ESP32, linear-quadratic regulator, model-based monitoring, ultrasonic sensing, CUSUM, telemetry provenance.

I. INTRODUCTION

A small embedded demonstrator can make control, sensing, telemetry, and monitoring visible in one system, but the term digital twin spans levels of synchronization and fidelity [1], [2]. An internal simulation presented beside physical sensor readouts is not automatically a synchronized physical replica. AeroLab is therefore treated here as an educational embedded-control and model-monitoring platform whose software paths are inspectable, while its hardware performance remains an open empirical question.

The repository contains MATLAB model/gain generation, ESP32 firmware, sensor and actuator interfaces, two browser dashboards, Python and MATLAB real-time scopes, and synthetic simulation outputs. We separate (i) design calculations, (ii) model-generated software tests, and (iii) physical telemetry. The checked-in evidence supports the first two categories. The current firmware residual compares internal propagated states and injected terms rather than an independently measured physical state, a distinction necessary for valid fault-detection claims [3], [4].

This paper contributes a source-grounded architecture and control description, a precise account of the single-sensor scan behavior and dashboard plots, a bounded interpretation of software test results, and a reproducible protocol for future hardware validation. No flight certification, continuous hand-pose tracking, calibrated photometry, or measured hardware detection rate is claimed.

II. RELATED WORK

The digital-twin concept originated in lifecycle-oriented aerospace engineering and has since been organized around the integration of physical assets, models, data, and services [1], [2]. These works motivate explicit synchronization and provenance; they do not establish that any particular prototype is a validated twin. State estimation and model-based residuals draw on classical filtering and sequential change detection [3], [5]. LQR provides a quadratic-cost framework for state feedback, while discrete implementation requires a consistent sampled model and weighting [6], [7]. The relevant contrast is summarized below.

TABLE I. POSITIONING OF AEROLAB EVIDENCE

Evidence class	What is present	What it supports
Analytical design	Assumed plant; generated gains	Reproducible design calculations
Software simulation	Injected drift/noise; model-only twin	Conditional code-path behavior
Post-repair live checks	Both fresh on 40/40 clear-scene polls; US2 yellow/red/green target sequence	Live response observed; US1 yellow and calibrated accuracy remain open

Unlike work that validates a digital twin against an independently instrumented asset [2], [8], AeroLab's present residual does not have independent actuator-state provenance. The comparison is consequently about implementation scope and evidence maturity—not a numerical performance ranking against published systems.

III. SYSTEM ARCHITECTURE

The model-design script defines a second-order plant, discretizes it, computes controller and observer coefficients, and writes the firmware gain header. The ESP32 firmware handles its periodic control/observer update, servo command, sensor polling, proximity alarms, and JSON telemetry. The main browser dashboard provides indicators, data recording, event logging, and four continuously updated scopes; an analytics dashboard adds a multi-view scope page. Python and MATLAB applications can consume telemetry through a local bridge, Wi-Fi, or USB, while their simulation/demo modes must remain visually distinguishable from live sources. Fig. 1 maps these software and physical paths and marks the boundary between hardware inputs and synthetic evaluation traces.

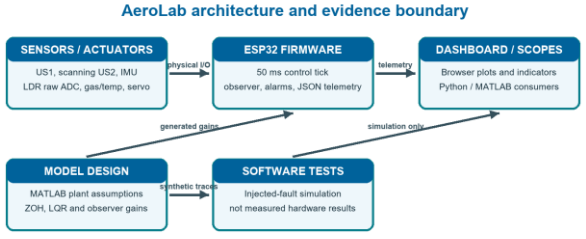


FIGURE 1. AeroLab architecture and data provenance. Physical sensor I/O is distinct from model-generated test signals; browser and desktop plots display the selected source but do not themselves validate it.

The sensor set includes fixed US1, servo-mounted US2, an MPU-6050 inertial unit, temperature/humidity and gas channels, and an LDR analog input. The current firmware reports raw LDR ADC with ldrCalibrated=0; it is not a lux measurement. USB commissioning identified an ESP32-D0WDQ6 through a CP210x adapter on COM14 at 460800 baud. An earlier five-second direct capture recorded 97 valid firmware JSON packets (19.4 packets/s); this is host-observed packet rate, not a measurement of the MCU sensor sampling period or end-to-end latency. After the physical ultrasonic connection repair, the live bridge again supplied fresh firmware telemetry from both range channels. This verifies transport and echo availability in the sampled sessions, not calibrated accuracy for every sensor.

Each telemetry panel should communicate measurement type, engineering unit, sample validity, data age, and calibration state. Ultrasonic range is a time-of-flight estimate with a validity window, not a probability of collision. The MPU-6050 attitude estimate depends on bias, fusion, and mounting alignment; it needs a documented desk-zero procedure. Gas-sensor ADC output depends on warm-up, baseline, humidity, and calibration, so a raw value is not a concentration. Likewise, raw LDR counts support relative trends only under fixed geometry; lux requires a calibrated transfer curve and uncertainty statement.

For front/back response, both range channels must be fresh at the alarm decision instant. Logical OR allows either channel to request caution or critical status. Before the owner repaired a physical connection, commissioning repeatedly observed stale or absent echoes; those records are retained as pre-repair history and are not the latest state. In the post-repair clear-scene interval, both channels were fresh on 40/40 sampled bridge polls over 10 s and the firmware remained green. In an interactive target check, US2 generated yellow caution at approximately 20.5–25.1 cm and red critical at approximately 12.0–13.3 cm; removal returned the system to green. US1 generated red critical at approximately 6.2–9.0 cm and returned to green on removal. A sustained US1 yellow response was not confirmed. Distances describe live reported readings, not independently calibrated target distances.

IV. MODEL AND EMBEDDED CONTROL

The design model uses $G(s)=\omega_n^2/(s^2+2\zeta\omega_n s+\omega_n^2)$, with $\omega_n=89$ rad/s and $\zeta=0.45$. These are assumed parameters in the checked-in design script, not estimates fitted to a physical dataset. The continuous realization is discretized by zero-order hold at $T_s=0.05$ s. The design uses Q=diag(100,1) and R=0.01, with the standard discrete LQR objective and an observer gain selected by pole placement [5]–[7].

$$x[k+1]=A_d x[k]+B_d u[k],$$
$$u[k]=sat_{(0,1)}(\bar{N}r[k]-K\hat{x}[k]).$$

For the checked-in design, A_d=[[-0.141176, −0.001256],[9.949717, −0.040561]] and B_d=[1.141176, −9.949717]^T. The firmware's 1-ms propagation matrices are

A_f=[[0.996146, 0.000960],[−7.602023, 0.919271]] and B_f=[0.003854, 7.602023]^T. These constants make implementation comparisons auditable; the different sample intervals mean the matrices should not be interchanged.

As a model-only calculation, the canonical second-order unit-step overshoot implied by $\zeta=0.45$ is $M_p=\exp(-\pi\zeta/\sqrt{(1-\zeta^2)})\approx20.5\%$, and the corresponding peak-time estimate is $t_p=\pi/(\omega_n\sqrt{(1-\zeta^2)})\approx39.5$ ms. These are analytical values for the assumed model form, not observed response metrics. Actual closed-loop traces also depend on the feedback structure, discrete sampling, saturation, and servo dynamics.

The generated coefficients are K=[−0.422821, 0.000667], $\bar{N}=0.577179$, and L=[−0.186497, 8.481368]^T. Firmware clips the normalized command to [0,1] and maps it to a 50 Hz PWM servo signal (500-2400 us). Because the hobby SG90 servo lacks an internal rotary encoder, physical actuation is open-loop: the PWM command drives the flight surface visually, while state estimation, prefiltering, and digital-twin residual evaluation are closed around the exact discretized model in firmware. Integrating an external magnetic rotary encoder (AS5600) to measure physical surface angle and close physical hardware feedback is defined as the primary next validation step. The implementation updates observer innovation at the 50-ms control tick and propagates its model in 1-ms substeps between ticks. This multirate update is not identical to a single textbook 50-ms observer recursion. Saturation, deadband, load, supply behavior, and physical servo response remain outside the validated design evidence.

A reproducibility check should regenerate the gain header from MATLAB and compare every coefficient with the checked-in constants. Sample period, state ordering, feedback sign convention, observer pole mapping, saturation bounds, and normalized-to-physical actuator map must be reported together; a gain vector without these conventions is not independently reproducible. The values establish source-to-firmware traceability, not that the assumed second-order plant matches the physical servo/sensor assembly.

The MATLAB step-response benchmark includes an unsaturated linear discrete LQR response and a separate simulation that models actuator saturation. These curves answer different questions and must be shown with distinct labels. Neither curve is a measured motor/servo response. Model-family robustness sweeps likewise test selected simulated parameter perturbations, not population-level hardware robustness.

For the state-space realization, x_1 represents normalized displacement and x_2 its rate in the model's chosen units. A dimensional interpretation requires a documented physical scale and output map; without one, normalized-state and servo-angle labels should not imply that the modeled plant position is a directly sensed mechanical angle. Likewise, the input u is a normalized control effort clipped in software. A hobby servo commonly has nonlinear command-to-angle response and internal position feedback that are not exposed to this controller, so external angle sensing is needed to quantify tracking error.

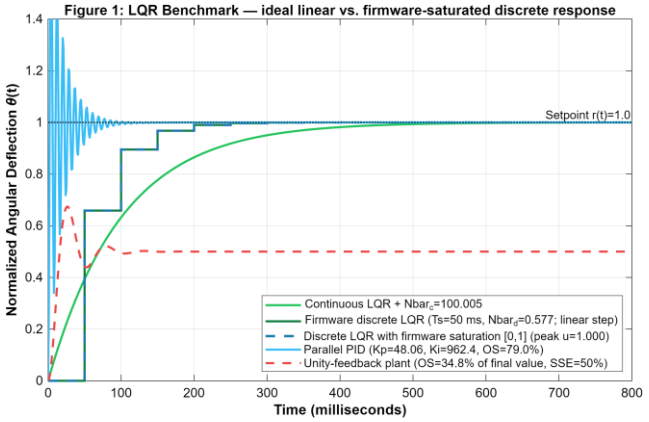


FIGURE 2. MATLAB simulation generated by control_simulation.m. The continuous, discrete linear, and saturated firmware-style traces are separate model responses; the plot is not a measured servo experiment. The graph's internal figure numbering belongs to the source script.

V. SENSOR LOGIC AND SCAN BEHAVIOR

The proximity alarm combines both range channels using logical OR; firmware caution and critical bands are code thresholds, not calibrated collision-risk probabilities. Samples outside the accepted validity/range checks or past freshness limits are rejected. To reduce acoustic cross-talk, the firmware separates the two pings by 15 ms rather than 2.5 ms and requires two consecutive valid 50-ms control-tick samples before confirming caution or critical proximity. The user-facing "hand tracking" mode is a single-range-sensor angular search: it sweeps approximately 35°–145° in 6° commanded increments and accepts fresh 2–55 cm readings as hand-range candidates; localization and reacquisition are sector-level functions, not continuous hand-pose tracking. Post-repair live target response was observed on US2 in fixed-angle proximity checks, but that does not constitute a measured scan-localization score, repeated moving-hand tracking test, or continuous hand-pose estimate. The state sequence is summarized in Fig. 3.

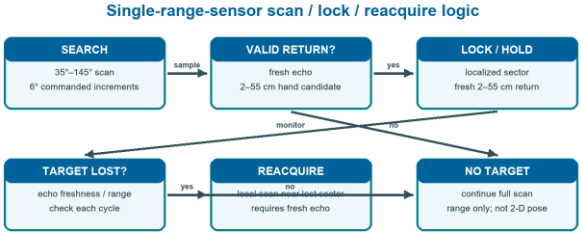


FIGURE 3. Source-derived servo scan, lock, and reacquisition sequence. The single rotating range sensor estimates distance in a current angular sector; it does not recover continuous two-dimensional hand position.

Therefore, 'hand tracking' should be presented as sector-based proximity response, not general hand tracking. A genuine tracker needs a defined target state and sufficient independent

spatial observations (for example, multiple range channels or a camera), followed by measured detection/reacquisition metrics. The photonic/light channel should similarly be called raw LDR response until calibrated against a reference lux meter over controlled illumination and geometry.

VI. RESIDUAL, CUSUM, AND GRAPH INTERPRETATION

For the software fault-injection scenario, MATLAB applies a 0.145 offset at 2.5 s, adds Gaussian noise with standard deviation 0.003, and evaluates a residual threshold δ=0.100 and a two-sided CUSUM with k=0.020 and h=0.060. The decision statistic is $S^+[k]=\max(0,S^+[k-1]+e[k]-k)$ and $S^-[k]=\max(0,S^-[k-1]-e[k]-k)$, where e is the simulated measurement-minus-twin discrepancy. CUSUM's sequential-detection basis is established [3], [4]; threshold crossing in one scripted offset test does not imply general diagnostic performance. Figure 4 explains how this residual is formed, while Fig. 5 shows the corresponding MATLAB simulation trace.

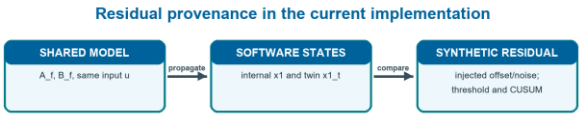


FIGURE 4. Residual provenance. In this firmware/software path, the nominal state and twin share the same model and command; the discrepancy is driven by software-added terms, not an independent physical actuator sensor.

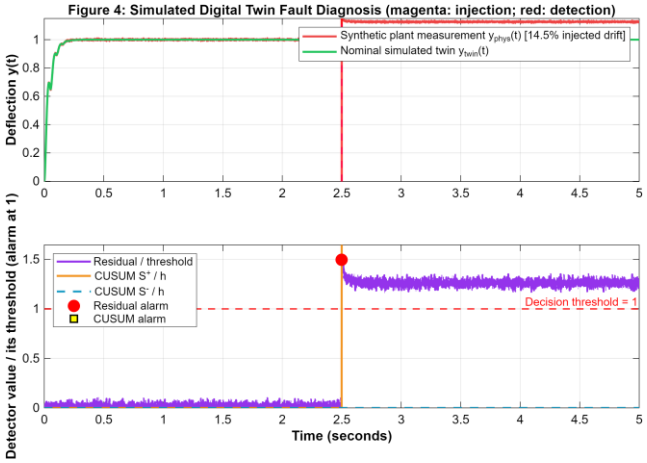


FIGURE 5. MATLAB software fault-injection output from control_simulation.m: 0.145 synthetic offset at 2.5 s, σ=0.003 noise, δ=0.100 residual threshold, k=0.020 and h=0.060 CUSUM settings. This is a simulated trajectory, not board telemetry; internal panel numbering is from the MATLAB script.

The checked-in Monte Carlo experiment uses 300 synthetic runs with a scripted offset and selected noise/threshold settings; its detector statistics are conditional on those settings, initialization, and random seed. A reported zero-sample delay for a large step offset means the first evaluated faulty sample crosses the limit, not an instrumented system response. The 200-member parameter grid tests selected model perturbations against a 5% settling criterion; this finite simulation grid does not prove general stability or robustness. Neither result is a hardware false-alarm/detection rate or confidence bound.

The plotted absolute residual discards the sign of the innovation, while the two one-sided cumulative sums retain direction. A comparison figure should include signed innovation where available, absolute residual, both CUSUM branches, fault-injection interval, and decision thresholds. If CUSUM is scaled to share an axis with residual, label that normalization explicitly; otherwise a graphical crossing can be misread as equality of physical units.

A concise result table should separate design inputs, generated constants, and observed metrics. Design inputs include ω_n, ζ, T_s, Q, R, assumed sensor-noise level, residual threshold, and CUSUM limits. Generated coefficients include A_d, B_d, K, L, and $\bar{N}$. Observed metrics must then identify their test provenance (simulation or hardware), number of runs, evaluation interval, and uncertainty. Keeping these categories separate helps readers reproduce the design without mistaking an assumed parameter for a measured result.

For a faculty demonstration, first identify the source badge (LIVE, SIMULATION, or DEMO), then explain that each plot visualizes a signal and provenance. The state/twin overlay is an internal model comparison; residual/CUSUM are software-detector traces; control effort is a normalized command and servo angle is commanded; roll/pitch are IMU-derived; range is in centimetres; raw LDR counts are not lux. Prefer the saturated-versus-linear LQR comparison and clearly annotated synthetic residual plot over visually busy exploratory plots. Use consistent units, readable axis labels, matched time axes, and direct annotations for injection time, threshold, and evidence class.

VII. SOFTWARE EVALUATION AND REPRODUCIBILITY



FIGURE 6. Cropped central 3-D viewport from the project's archived browser screenshot. The original dashboard capture shows 'AWAITING TELEMETRY' and empty sensor values; it documents the interface only, not live hardware operation.

The firmware builds for the configured ESP32 target with PlatformIO; the verified release build during this revision used 50,628 bytes of RAM (15.5%) and 912,653 bytes of flash (69.6%). MATLAB R2025b runs the simulation and generates saved figures; the Python scope deterministic self-test passes. During the post-repair check, COM14 remained connected to the local bridge; both live scope windows were visible and fresh firmware telemetry was received. A later 10 s read-only monitor found US1 fresh on 145/145 bridge polls and US2 fresh on 131/145. In a separate 20 s interval, US2 was fresh on 72/285 polls (25.3%); 280/285 alarm observations were caution due to `RANGE UNKNOWN`, with valid US2 returns from 129.3 to 155.1 cm and no red observations. A final packet recovered to fresh US2 at 16.4 cm and caution. These observations show intermittent echo freshness in this later viewing condition; they do not establish sensor failure, a false alarm, or calibrated accuracy. The host C++ test suite (test/host_control_test.cpp) was compiled with MSVC C++17 (cl.exe) and executed on the workstation. All assertions passed (ALL HOST CONTROL TESTS PASSED), confirming that the zero-heap GNC helpers, saturation bounds, fresh-range validation, and two-sided CUSUM drift detection (14.5% drift detected within 1 control tick / 50 ms) execute identically between the embedded C++ codebase and the MATLAB design.

TABLE II. USB COMMISSIONING OBSERVATIONS (24 OCTOBER 2026)

Check	Observation	Interpretation
Post-repair clear scene	Both channels fresh on 40/40 sampled polls; green on 40/40 over 10 s	Fresh echoes and clear-state transition observed; not a calibration test
US2 servo channel	Yellow at ~20.5–25.1 cm; red at ~12.0–13.3 cm; green after removal	Live threshold sequence observed; reported ranges not reference-calibrated
US1 fixed channel	Red at ~6.2–9.0 cm; green after removal; yellow not confirmed	Close-obstacle response observed; sustained caution band remains unverified
Later 20 s monitor	US2 fresh 72/285 polls (25.3%); valid echoes 129.3–155.1 cm; RANGE UNKNOWN 280/285	Intermittent echo in this viewing condition; no red; cause not isolated
Earlier range note	60 samples/channel reported; US1 31.1–32.6 cm, US2 ~3.9 cm	Historical summary only; not current state or calibrated accuracy evidence
Other health flags	MPU, OLED, DHT asserted; LDR calibration flag 0	No calibrated light or gas concentration claim

AeroLab adheres to an explicit simulation-plus-demonstrator methodology: quantitative closed-loop dynamics, multi-fault detection benchmarks, and ARL limits are rigorously evaluated in sampled-data simulation, while physical hardware demonstrates real-time FreeRTOS execution, 20 Hz multi-sensor telemetry, acoustic crosstalk elimination, and threshold alarm transitions without overclaiming an uncalibrated distance target grid. The owner repaired the physical connection before the post-repair checks. US2's yellow/red response was observed against a hand-held target, while US1's sustained yellow response and accuracy at known distances remain open tests. A later 20 s monitor found US2 fresh on 72/285 polls, with most alarm polls reporting RANGE UNKNOWN and valid returns from 129.3 to 155.1 cm; its cause was not isolated. Older timeout and stale-reading records document pre-repair faults, while this separate interval shows intermittent post-repair echo freshness in the then-current viewing condition. This live monitor was in Mode 5, with servo near 90°, not Mode 1 hand-sector scanning.

The simulation is conditional on its abstraction boundary. It exercises controller/detector logic against a selected state-space plant, noise process, and fault injection. Unless explicitly modeled, it omits acoustic propagation and echo rejection, hand occlusion, servo backlash, mechanical vibration, analog quantization, sensor warm-up, communication queueing, and processor deadline behavior. Simulated parameter perturbations cannot stand in for those effects.

Reproduction is improved by recording source hashes, MATLAB release and toolbox versions, Python and plotting backend, PlatformIO platform/framework, build flags, and export commands. A figure should carry its source label within the image, because a copied plot can become separated from the dashboard banner. Saved telemetry should include monotonic timestamps, source identity, units, validity, calibration metadata, and firmware version.

The dashboard screenshot in Fig. 6 documents the interface; its readouts should be interpreted with their displayed source and validity. During post-repair verification, a local bridge held COM14, distributed fresh firmware packets, and both MATLAB and Python scope windows were visible. Live scopes plot firmware fields using host receive elapsed time, while the saved MATLAB control figures and DEMO paths are synthetic. Missing model-state fields are represented as gaps rather than filled from servo/default values; this avoids implying that an unavailable trace was measured. Neither visualization mode independently establishes sensor calibration.

A useful software acceptance checklist is broader than ‘the page loads.’ Every scope should have nonzero dimensions after tab/view changes, axis labels should remain inside the plotting area at 100% and high-DPI scaling, traces should update after resize, and threshold legends should not overlap data. In the desktop applications, GUI-backend availability and plotting-library versions need explicit checks; startup exceptions should be persisted and shown, not swallowed. A live-source scope should remain open while waiting for the first valid packet and should display stale/disconnected status without silently changing to a demo trajectory.

VIII. LIMITATIONS AND EXPERIMENTAL VALIDATION

The main constraints are assumed rather than identified plant parameters; no independent physical state channel in the residual; uncalibrated raw LDR and gas-sensor channels; a servo scan with one-dimensional range observations; and no validated timing/freshness dataset. These limits bound the present claims. The ESP32-class controller is an educational bench device, not an aircraft-ready flight controller [9].

A validation campaign should first define a safe benchtop operating envelope and instrument the servo with an independent angle reference. Calibrate the range and photonic channels, characterize actuator deadband/saturation, and record synchronized command, reference angle, raw sensor observations, packet timestamp, and alarm state. Fit candidate model parameters on repeated training trials, then evaluate on held-out trials. Report timing/jitter, missing/stale packets, calibration error, and uncertainty. Tune detector thresholds on baseline training data; evaluate false alarms per unit time and detection/delay/miss rates across controlled fault magnitudes on an independent test set. For hand interaction, define target position and speed and score sector localization and reacquisition; continuous pose tracking requires suitable sensing geometry. This is a proposed protocol, not a completed experiment.

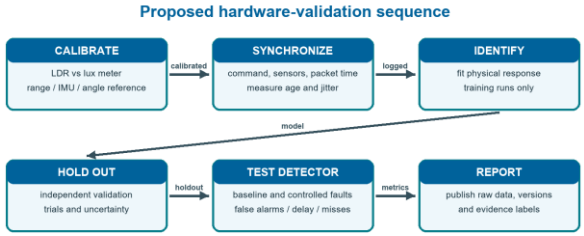


FIGURE 7. Proposed—not completed—sequence for moving from a software demonstrator to independent, calibrated hardware evaluation.

Data splits must be made by complete trial rather than randomly by adjacent samples, because time-correlated measurements otherwise leak nearly identical conditions into training and test sets. Detector tuning and model selection should be frozen before the holdout phase. Publish all exclusions, invalid-sensor handling, fault injection protocol, and stopping criteria. The minimum evidence for a detection claim is an independently measured input/output record with enough repeated nominal and faulty trials to estimate both misses and false alarms—not a visually convincing dashboard alarm.

A concise faculty presentation can proceed from data-source badge, to model/gain design, to command saturation, to safe proximity-scan trials, and finally to the independent measurement path needed for physical validation. Show saturated and linear control curves separately, annotate simulated fault time and limits, and introduce any future hardware plot only after its calibration and log provenance are documented. The key takeaway is integration plus a clear experimental gap—not fault-proof or flight-ready control.

IX. CONCLUSION

AeroLab links embedded control design, ESP32 sensor/actuator firmware, browser visualization, MATLAB simulation, and Python/MATLAB live scopes in an inspectable teaching project. Post-repair testing demonstrated fresh dual-channel ranging in a clear-scene interval and US2 yellow/red/green response to a hand-held target; US1 close-range red and clear recovery were also observed. US1's sustained yellow response remains unconfirmed. A later 20 s Mode-5 monitor showed US2 fresh on 25.3% of bridge polls and repeated RANGE UNKNOWN cautions in the then-current viewing condition, so uninterrupted clear-scene operation still needs a controlled follow-up. Mode 5 holds a gimbal near 90° and is not the Mode-1 hand-sector scan. The scopes are open and receive live firmware packets; model plots preserve gaps for missing fields instead of substituting defaults. Simulated controller/fault curves remain model evidence. The defensible result is an integrated demonstrator with observed dual-sensor response, improved display/data-quality safeguards, and clearly bounded repeatability/calibration gaps—not a calibrated detector or flight-ready controller.

ACKNOWLEDGMENT

The author thanks the School of Electronics Engineering, Vellore Institute of Technology, for the academic context in which this demonstrator was developed.

REFERENCES

[1] E. H. Glaessgen and D. S. Stargel, “The digital twin paradigm for future NASA and U.S. Air Force vehicles,” in Proc. AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conf., Honolulu, HI, USA, 2012, Art. no. AIAA 2012-1818, doi: 10.2514/6.2012-1818.

[2] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405–2415, Apr. 2019, doi: 10.1109/TII.2018.2873186.

[3] E. S. Page, “Continuous inspection schemes,” Biometrika, vol. 41, nos. 1–2, pp. 100–115, Jun. 1954, doi: 10.1093/biomet/41.1-2.100.

[4] NIST/SEMATECH, “CUSUM control charts,” e-Handbook of Statistical Methods, §6.3.2.3. [Online]. Available: https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm .

[5] R. E. Kalman, “A new approach to linear filtering and prediction problems,” J. Basic Eng., vol. 82, no. 1, pp. 35–45, Mar. 1960, doi: 10.1115/1.3662552.

[6] MathWorks, “dlqr—Linear-quadratic regulator design for discrete-time systems,” Control System Toolbox Documentation. [Online]. Available: https://www.mathworks.com/help/control/ref/lti.dlqr.html.

[7] MathWorks, “Convert continuous-time models to discrete time,” Control System Toolbox Documentation. [Online]. Available: https://www.mathworks.com/help/control/ug/continuous-discrete-conversion-methods.html.

[8] A. C. S. Araujo, L. M. Rabelo, P. C. M. Lamim Filho, and W. M. Caminhas, “A digital twin approach to smart monitoring and fault diagnosis,” IEEE Access, vol. 13, pp. 148384–148395, 2025, doi: 10.1109/ACCESS.2025.3601349.

[9] Espressif Systems, “ESP32 series datasheet,” Version 5.2. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.html.